Nimbus Platform

Production-Ready Cloud Architecture

Created by: Cielo Escobar

What is this project?

This project is a production-ready cloud platform designed to simulate how a real application would run in a scalable and reliable environment.

- Built on AWS using Terraform
- Designed with real-world patterns
- Focused on infrastructure, not application complexity

Project Goal

The goal was not just to deploy infrastructure, but to design a system that reflects how production environments are actually built.

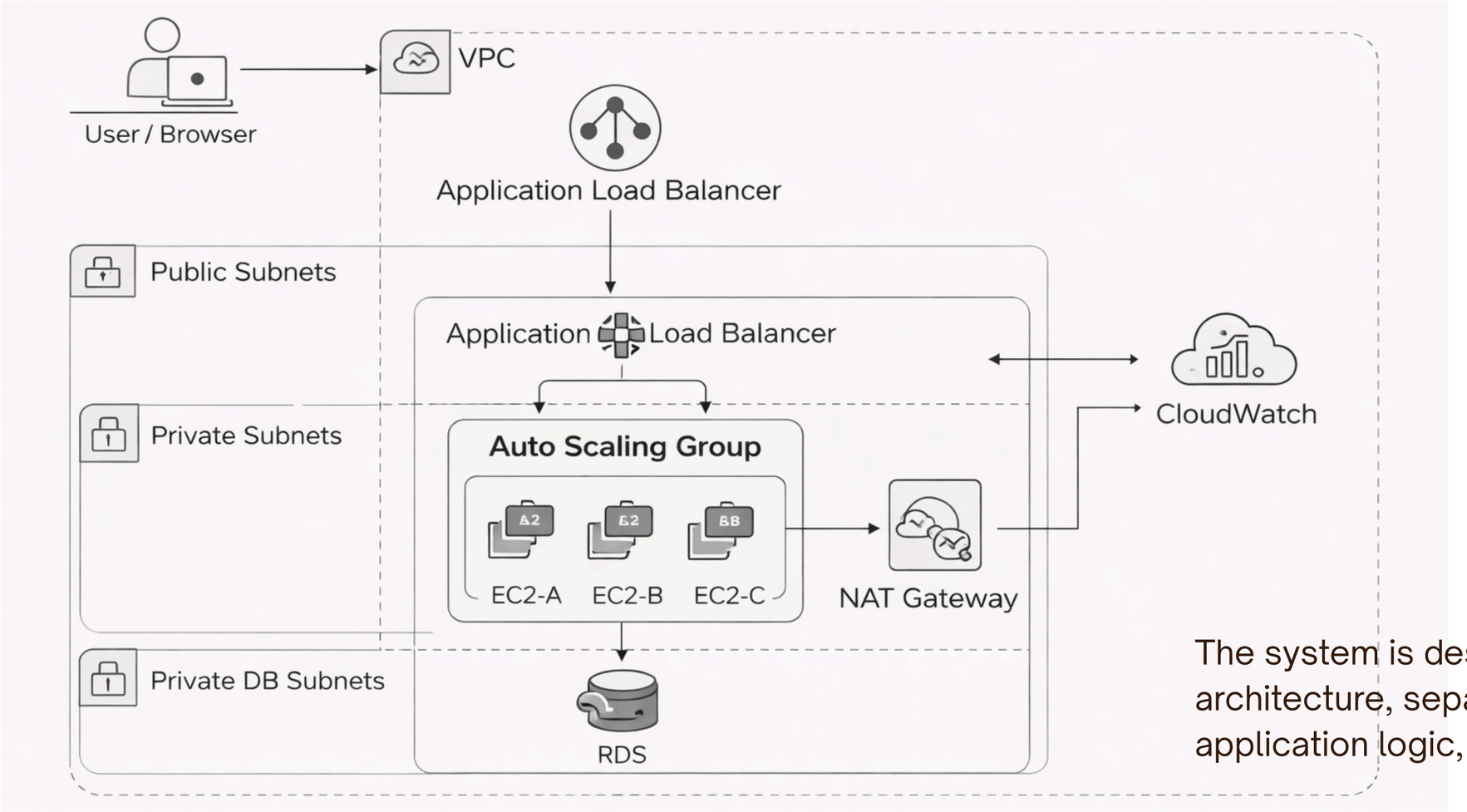
- High availability across multiple zones
- Automated deployment using CI/CD
- Monitoring and alerting from the beginning

My Approach

Instead of adding complexity, I focused on building a solid foundation first and validating each layer before moving forward.

- Start simple, then iterate
- Validate every component
- Make decisions based on trade-offs

Architecture Overview



The system is designed as a multi-tier architecture, separating public access, application logic, and internal resources.

Key Architecture Decisions

Every design choice was made to balance simplicity, reliability, and cost.

- Multi-AZ → improves availability
- Private subnets → reduce exposure
- Stateless app → enables scaling
- Terraform modules → keep structure clean

nimbus-dev-app	i-0521f7355130d21a8	Running	t3.micro	3/3 checks passed	View alarms +	eu-central-1a
nimbus-dev-app	i-07098a3bebf7e83b8	Running	t3.micro	3/3 checks passed	View alarms +	eu-central-1b
nimbus-dev-app	i-05f1d55b6861d90a6	Running	t3.micro	3/3 checks passed	View alarms +	eu-central-1b

Networking Strategy

The network is designed to isolate components while still allowing controlled communication between layers.

- Public layer: Load Balancer
- Private layer: Application instances
- NAT Gateway enables outbound access only

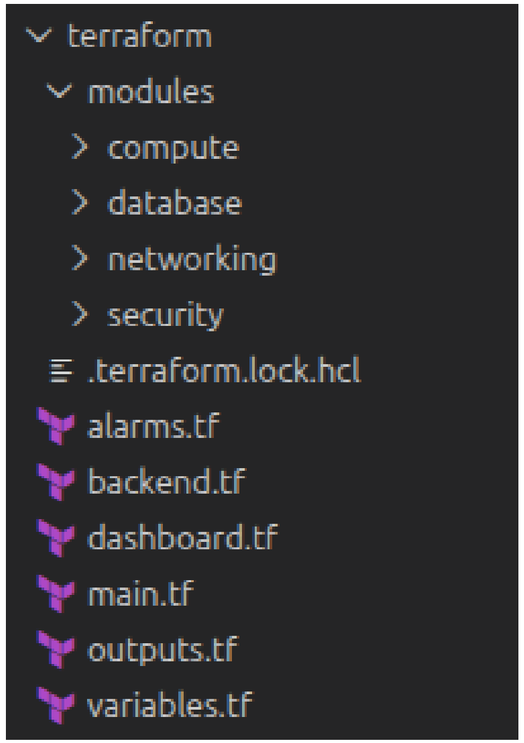
Name	Subnet ID	State	VPC	IPv4 CIDR
nimbus-dev-private-app-1	subnet-0d6bd47ea8bb2df88	Available	vpc-03afb90bc3b90b1a2 | nim...	10.0.11.0/24
nimbus-dev-private-app-2	subnet-01d4cf6721563120a	Available	vpc-03afb90bc3b90b1a2 | nim...	10.0.12.0/24
nimbus-dev-private-db-1	subnet-02ac64a21a1b0a5f3	Available	vpc-03afb90bc3b90b1a2 | nim...	10.0.21.0/24
nimbus-dev-private-db-2	subnet-05fccc95c9db7d693	Available	vpc-03afb90bc3b90b1a2 | nim...	10.0.22.0/24
nimbus-dev-public-1	subnet-0a473abb71916fdc6	Available	vpc-03afb90bc3b90b1a2 | nim...	10.0.1.0/24
nimbus-dev-public-2	subnet-051ef958ff4aff370	Available	vpc-03afb90bc3b90b1a2 | nim...	10.0.2.0/24

Application Design

The application is intentionally simple, but structured to behave like a real production service.

- Stateless Flask app
- Health endpoint for monitoring
- Displays instance and AZ for visibility

Infrastructure as Code



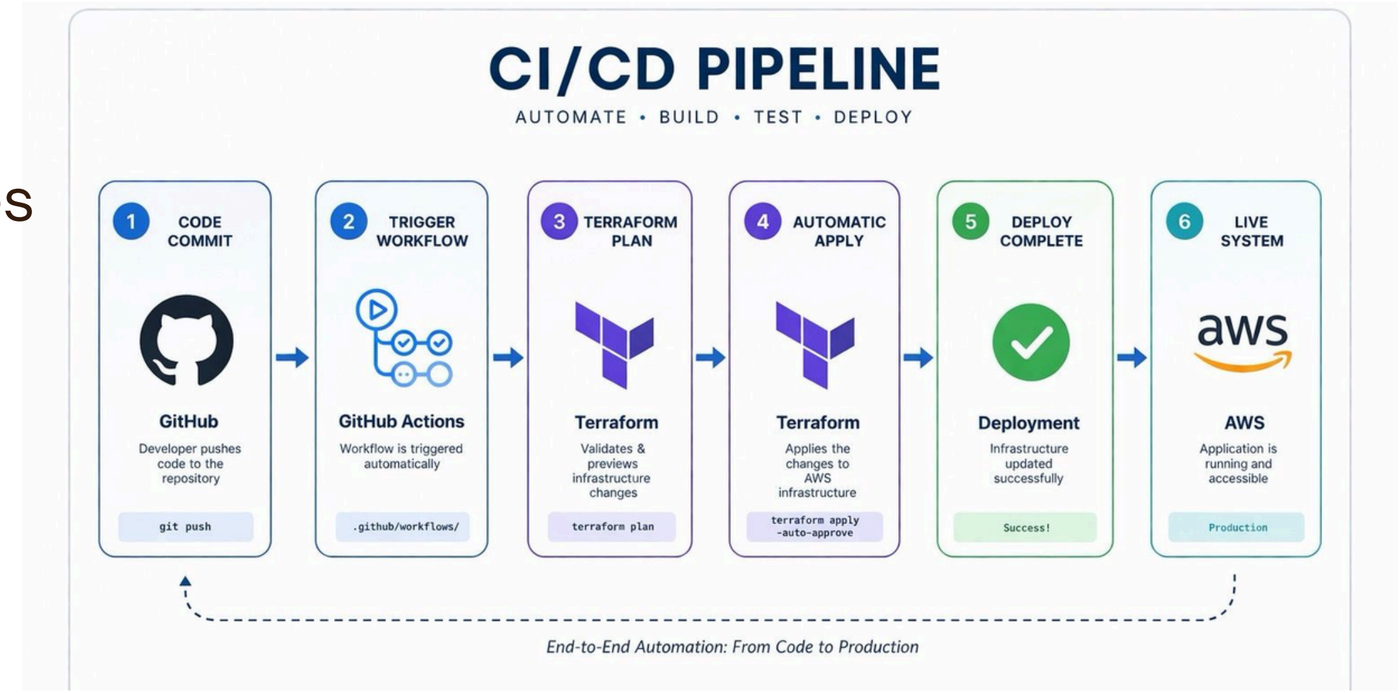
Infrastructure is fully defined using Terraform, allowing reproducibility and version control.

- Modular structure (networking, compute, security)
- Environment-ready setup
- Remote state configuration

CI/CD Pipeline

Infrastructure changes are managed through a GitOps workflow, ensuring controlled and automated deployments.

- PR triggers validation and plan
- Merge triggers deployment
- No manual infrastructure changes



← Terraform Checks

Add files via upload #21

Summary

All jobs

Terraform Format and Validate

Run details

Usage

Workflow file

Triggered via push 42 minutes ago

Cielo-e997 pushed c1911f0 main

Status

Success

checks.yml

on: push

Terraform Format and Vali... 17s

← Terraform Checks

Add files via upload #21

Summary

All jobs

Terraform Format and Validate

Run details

Usage

Workflow file

Triggered via push 42 minutes ago

Cielo-e997 pushed c1911f0 main

Status

Success

checks.yml

on: push

Terraform Format and Vali... 17s

← Terraform Apply

cleanup: remove unused files and fix names #15

Summary

All jobs

Terraform Apply

Run details

Usage

Workflow file

Triggered via push 2 hours ago

Cielo-e997 pushed 5beaba8 main

Status

Success

terraform-apply.yml

on: push

Terraform Apply 25s

Monitoring Strategy

Monitoring focuses on visibility of system health rather than just collecting metrics.

- CPU, latency, errors, traffic
- Centralized CloudWatch dashboard
- Designed for quick issue detection

Alerting Approach

Alarms (6)

Clear selection

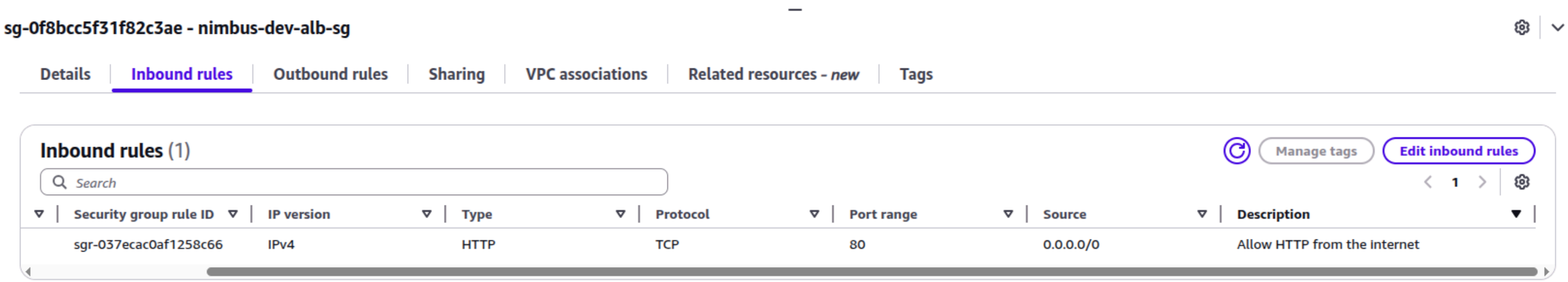
Filter alarms | Quick filters

Name	State	Conditions	Metric name
nimbus-alb-unhealthy-hosts	OK	UnHealthyHostCount > 1 for 2 datapoints within 2 minutes	UnHealthyHostCount
nimbus-high-cpu	OK	CPUUtilization > 70 for 2 datapoints within 2 minutes	CPUUtilization
nimbus-alb-5xx-errors	Insufficient data	HTTPCode_ELB_5XX_Count > 1 for 1 datapoints within 1 minute	HTTPCode_ELB_5XX_Count

Alerts are configured to be actionable, avoiding unnecessary noise.

- Focus on critical failures
- Thresholds tuned to avoid false positives
- Designed to support quick response

Security Approach



Security is applied at multiple layers, following a defense-in-depth strategy.

- Network isolation between tiers
- IAM roles with minimal permissions
- No secrets stored in code

Cost Strategy

Cost decisions were made consciously, balancing performance and efficiency.

- Small instance types
- No over-provisioning
- Clear monthly cost estimation

Cost Breakdown (Monthly Estimate)

Service	Usage / Configuration	Estimated Monthly Cost
EC2 (Auto Scaling Group)	3 × t3.small (720h)	~$45
RDS (PostgreSQL)	db.t3.micro (Single-AZ)	~$15
Application Load Balancer	1 ALB + low traffic	~$16
NAT Gateway	1 NAT + low data processing	~$35
EBS Storage	~100 GB	~$10
CloudWatch	Logs + Metrics + Alarms	~$8
	Total Estimated Monthly Cost	~$120 – $140

Trade-offs

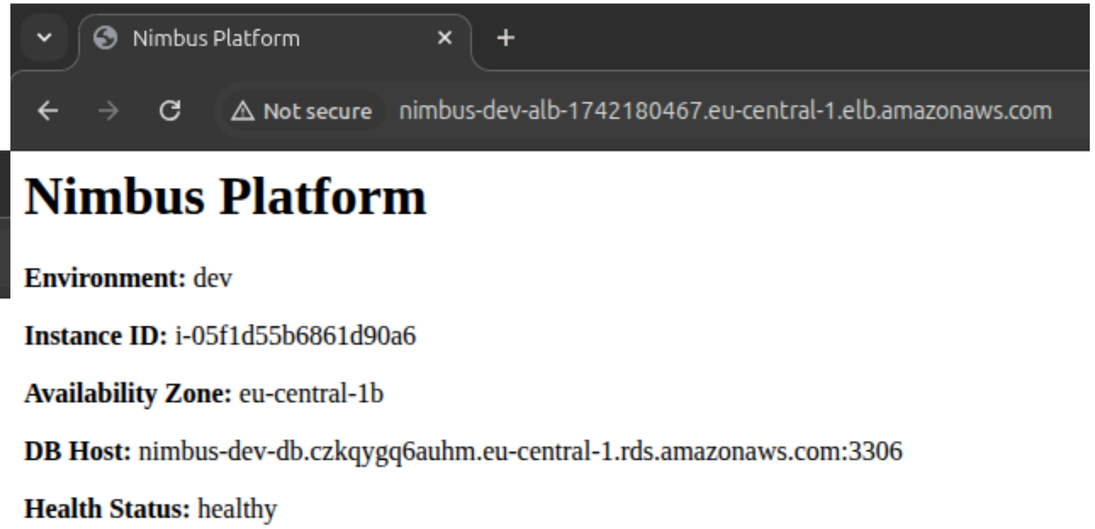
Throughout the project, several trade-offs were necessary.

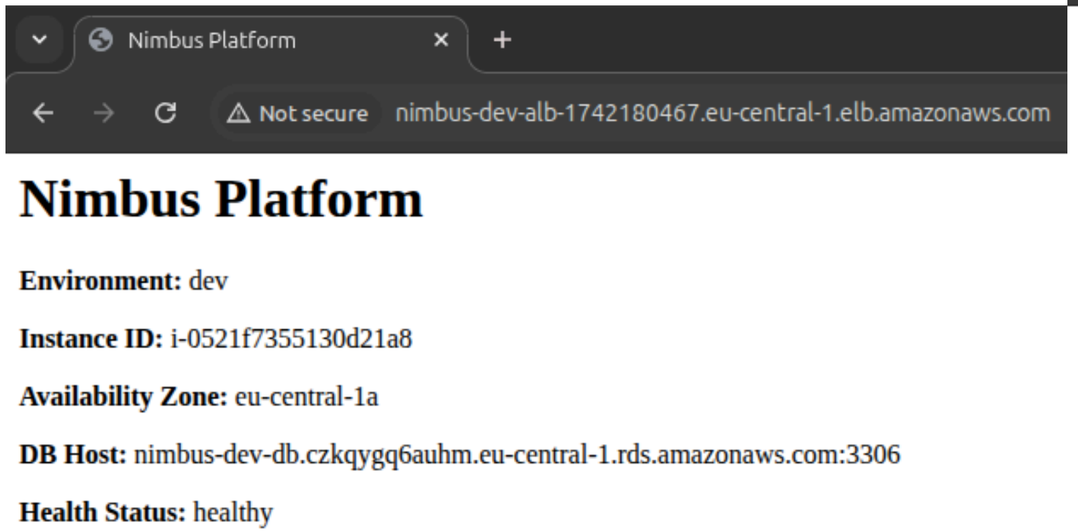
- Simplicity over complexity
- Cost over maximum resilience
- Speed of delivery over feature completeness

Demo

This is the system running in a real environment.

- Application accessible publicly
- Multiple instances behind load balancer
- Monitoring dashboard active





Challenges & Solutions

Building this system involved solving real-world problems.

- Networking issues → fixed with security group adjustments
- Terraform errors → improved module structure
- Monitoring noise → refined alert thresholds

Future Improvements

With more time, the platform could evolve into a more advanced system.

- Add database layer
- Improve autoscaling policies
- Add security layers like WAF
- Introduce infrastructure testing

Final Takeaway

This project reflects how I approach building reliable and scalable cloud systems.

- End-to-end infrastructure
- Real-world practices applied
- Strong focus on decisions, not just tools